

Group: Gradient Dissent

Members: John Jezl, Jonathan Rocha, Janice Bargariora, Gregory Demo

Project: Rubik's Cube Solver

Github Link: <https://github.com/johnjezl/CS-480-Signature-Assignment>

Responsibilities:

John Jezl: Color Classification

Jonathan Rocha: Computer Vision Segmentation

Janice Bargariora: Hardware Setup

Gregory Demo: Solving Algorithm

Introduction:

Our project is a program that looks at a Rubik's Cube and lists the moves to solve it. Rubik's Cubes provide a unique computer vision challenge because a cube holds so much information. Our algorithm needs to segment 54 facelets across 6 images. Segmentation and classification are the hardest part of this project. Solving algorithms have been well researched and are easy to implement, but they require you to create what is essentially a Rubik's Cube simulation that can perform moves.

Pipeline:

1. The user captures images of each face of the Rubik's cube (camera on Jetson, or file-based on any platform).
2. A segmentation algorithm extracts bounding boxes for each of the 9 facelets.
3. A CNN color classifier labels each facelet with confidence scores.

4. The cube state is validated, with automatic orientation correction using edge constraints.
5. The cube is solved using Kociemba algorithm.
6. The solution is displayed via 3D animation with human-readable move instructions

Hardware:

A Jetson Nano with an attached IMX219 camera was our choice of target device. Although the program can accept images from the file system and runs cross-platform (Windows, macOS, Linux), we wanted something more physical to match the Rubik's Cube. The Jetson provides GPU acceleration via NVIDIA VPI for faster preprocessing. We created a little stage with a background to help take consistent pictures.

Facelet Segmentation:

In our computer vision algorithm, we first segment the bounds of the Cube. By using diverse sets of images early in our testing, we noticed that the background could create many edge cases. If the background had straight lines in it, then the algorithm could get confused. A more insidious case is when certain facelet colors blend into the background when they are all along the same edge. We developed 5 segmentation algorithms to handle different scenarios: Grid Division for centered cubes, Contour-Perspective for tilted cubes, Contour-Neighbor (our default and most reliable), Canny-Square for high-contrast images, and Brightness-Otsu for varied lighting. An adaptive evaluation system selects the best algorithm and preprocessing combination

based on historical metrics. Each successfully segmented face yields 9 facelets of 64x64 pixels.

Color Classification:

For the system to work, we need to know which of the six possible colors each facelet is. Algorithmic approaches didn't seem to generalize well, so we trained a CNN model. Training takes many images and we didn't have the time or motivation to take a bunch ourselves. The internet didn't have labeled data of Rubik's Cube stickers, so using synthetic data is our final option. We built a generator that creates facelet images under different lighting conditions and automatically labels them. The CNN is a lightweight 3-block convolutional network with batch normalization and dropout, outputting confidence scores for all 6 colors. Red and orange are the most commonly confused, but Blue is the hardest to detect overall.

Cube State:

Storing a color for each of the 54 stickers on a Cube is cumbersome. We encode corner and edge pieces in their own arrays. Each element of the array is a position on the Cube. The element holds the position the piece is supposed to be and how far it is rotated from being in the correct orientation. Using this representation, we can easily implement the core 6 moves of the cube.

Solving Algorithm:

Human methods of solving a Rubik's Cube can take just a few seconds to execute, but that is because people have worked very hard to perform moves quickly. It would be terrible if our program displayed a string of 60 moves, so we must find strategies that prioritize move count. In the search for "God's Number," the maximum

number of moves needed to solve from any cube state, Herbert Kociemba developed a two phase algorithm for computing a solution to a cube. The first phase gets the cube into a convenient state while the second phase uses a reduced move set to finish the solution.

Results:

We created a set of test data which included a picture of each face from 10 random scrambles. The program was able to output a set of moves for each of these 10 sets. To ensure these moves solve the cube, we performed the moves backwards on our actual cubes and then compared the physical cube state to the images. It worked every time!